AWS VPC Peering: İki VPC’yi Özel Hatla Birleştir!

AWS üzerinde büyüyen sistemlerde işler genellikle tek bir VPC ile sınırlı kalmaz. Uygulamalar farklı bölgelerde çalışabilir, bazı bileşenler public erişime açıkken bazıları tamamen private kalabilir. İşte tam bu noktada **VPC Peering**, iki VPC’yi birbirine özel bir hat üzerinden bağlamamızı sağlar.

Neden VPC Peering?

- **Düşük gecikme**: Trafik AWS backbone üzerinden akar, internetten dolaşmaz.
- **Güvenlik**: Bağlantı tamamen private IP’lerle çalışır.
- **Basitlik**: VPN ya da Transit Gateway kurmaya gerek kalmadan iki VPC’yi konuşturabilirsiniz.

Ama unutmayın:
⚠️ VPC Peering’de **transit routing yoktur**. Yani A→B, B→C bağlıysa A doğrudan C’ye gidemez.
⚠️ CIDR bloklarının çakışmaması gerekir.
⚠️ Route tabloları **her iki tarafta da** güncellenmelidir.

🎯 **Bu yazıda ne öğreneceksiniz?**

Bu lab çalışmasında:

- Aynı bölgede (örneğin **us-east-1 N. Virginia**) iki VPC kuracağız.
  - **VPC-A:** Public subnet içeriyor, bastion EC2 çalışıyor.
  - **VPC-B:** Private subnet içeriyor, public erişim yok.
- Bu iki VPC arasında **peering connection** oluşturacağız.
- Sonrasında bastion EC2 üzerinden private EC2’ya **özel IP ile SSH bağlantısı** kuracağız.
- Lab sonunda kaynakları silip maliyet sürprizlerini önleyeceğiz.

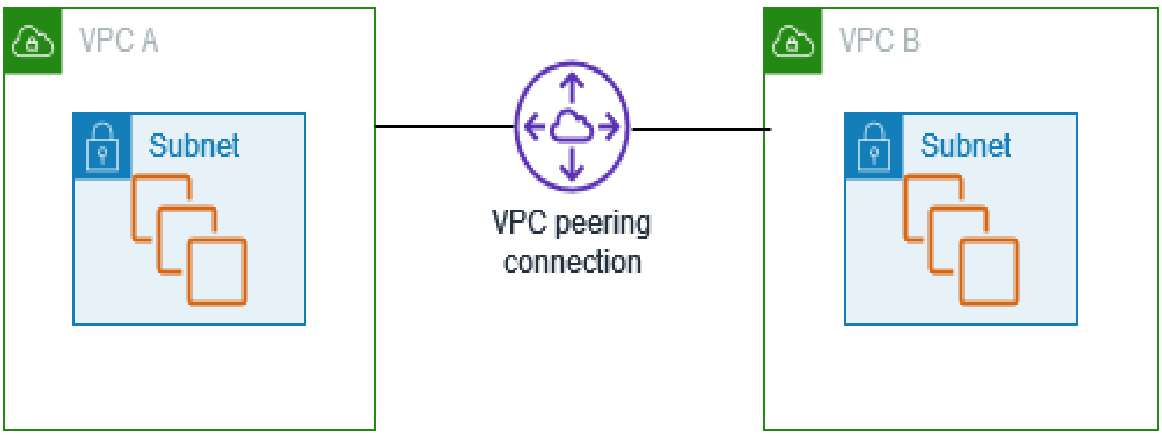
🗺️ Senaryo ve Mimari

- **VPC-A (Public)**
  - **CIDR:** `10.0.0.0/16`
  - **Public Subnet:** `10.0.1.0/24`
  - İçinde çalışan EC2: VPC-A_EC2
    - Public IP var
    - Bastion görevi görüyor
    - SSH + HTTP erişimi açık
- **VPC-B (Private)**
  - **CIDR:** `20.0.0.0/16`
  - **Private Subnet:** `20.0.0.0/24`
  - İçinde çalışan EC2: VPC-B_EC2
    - Public IP yok
    - Sadece içeriden SSH erişimi

👉 **Hedef:** VPC-A'daki bastion EC2 üzerinden, VPC-B'deki private EC2'ya özel IP ile SSH bağlantısı kurmak.

Mimariyi Basitçe Hayal Edelim

- VPC-A → Public Subnet → Bastion EC2 (internet erişimli)
- VPC-B → Private Subnet → Private EC2 (yalnızca internal IP)
- Aralarında **VPC Peering Connection** var.
- Route tabloları karşılıklı güncellenmiş durumda.



VPC-A (public subnet) ile VPC-B (private subnet) arasında kuracağımız VPC peering bağlantısının mimarisi

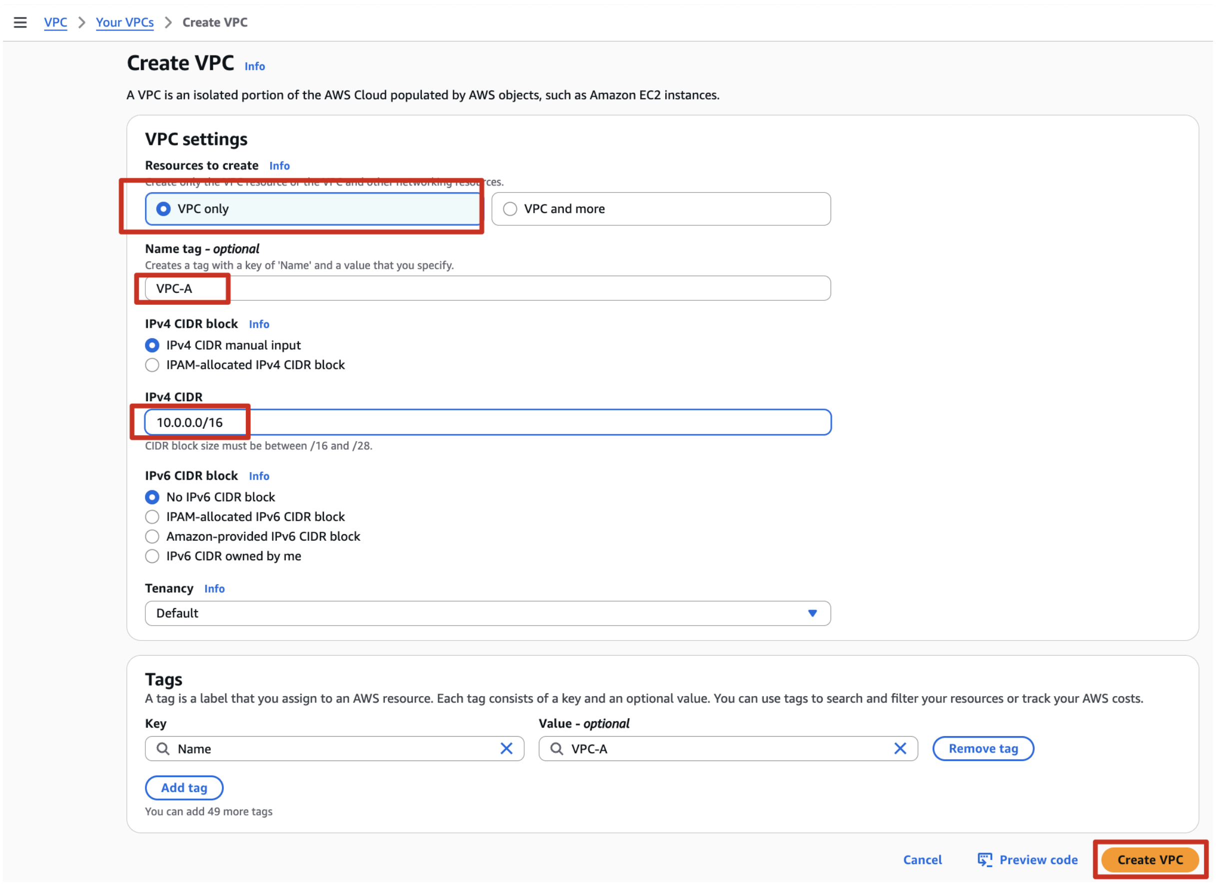
Bölüm 1 — VPC-A (Public) Kurulumu

İlk olarak internet erişimi olan **VPC-A**'yı oluşturacağız. Bu VPC içinde bir **public subnet** kuracak, internet erişimi için **Internet Gateway** ekleyecek ve en sonunda bir **bastion EC2** çalıştıracağız. Bu bastion sunucusu üzerinden diğer VPC'ye bağlanacağız.

1. VPC Oluşturma

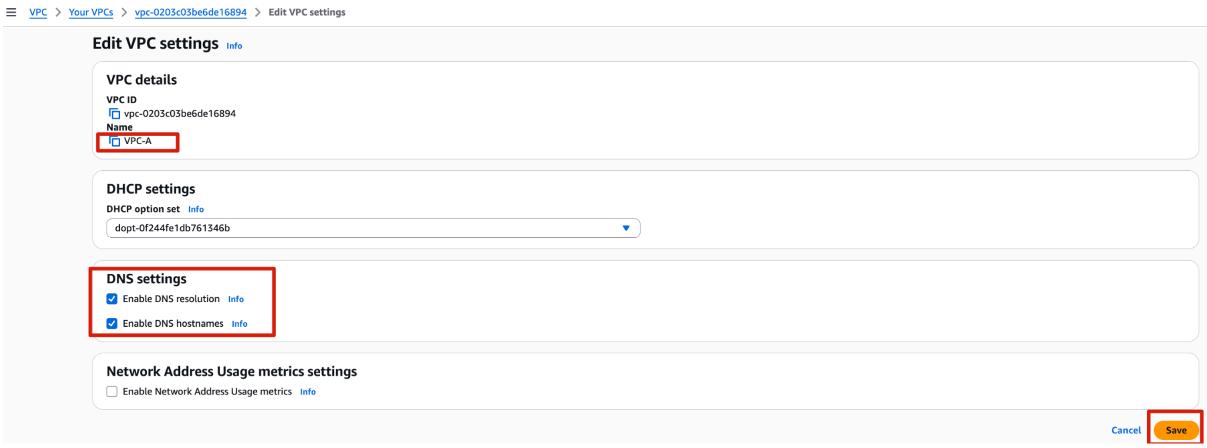
VPC→Your VPC→Create VPC

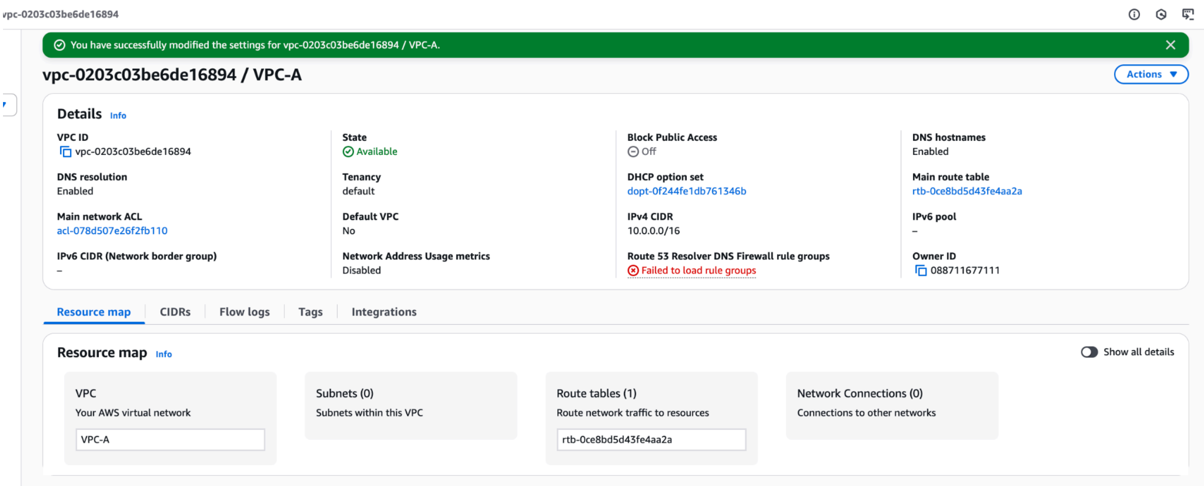
- AWS Console'a giriş yapın.
- Bölge olarak **N. Virginia (us-east-1)** seçin.
- Menüden **VPC → Your VPCs → Create VPC** yolunu izleyin.
- **Seçenek:** *VPC only*
- **Name:** `VPC-A`
- **IPv4 CIDR:** `10.0.0.0/16`
- Diğer ayarları varsayılan bırakıp **Create VPC** butonuna basın.



Listeden **VPC-A**’yı seçin → **Actions** → **Edit VPC settings**.

- **Enable DNS resolution** ve **Enable DNS hostnames** kutularını işaretleyip **Save edin**

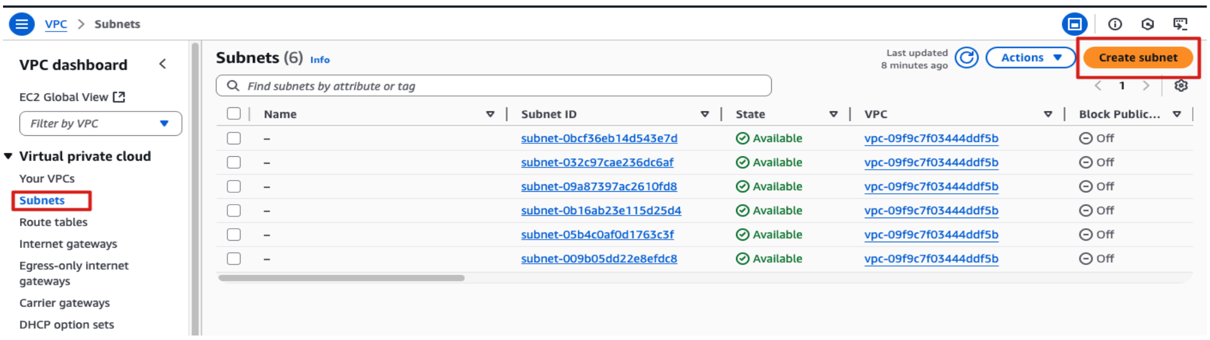




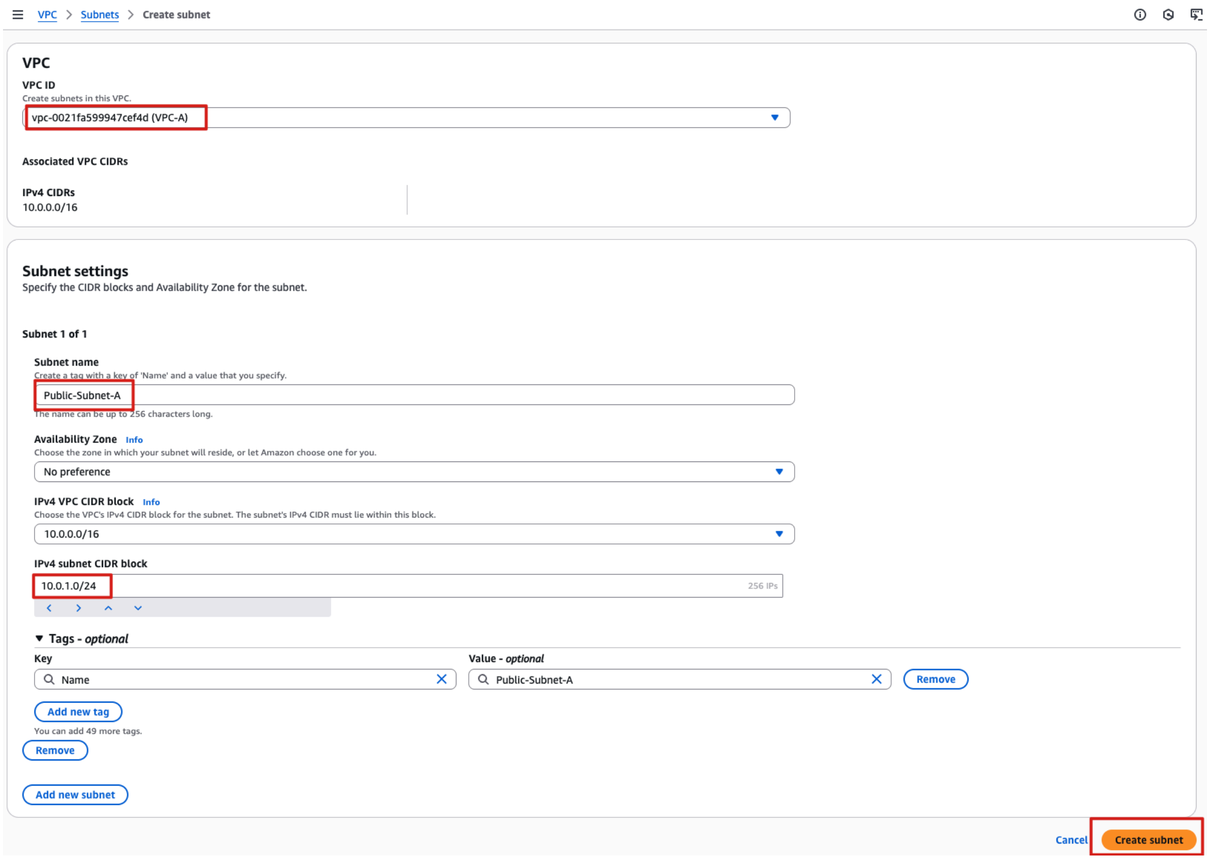
✅ **Artık VPC-A hazır. DNS resolution ve hostnames ayarları aktif.**

2. Public Subnet Oluşturma

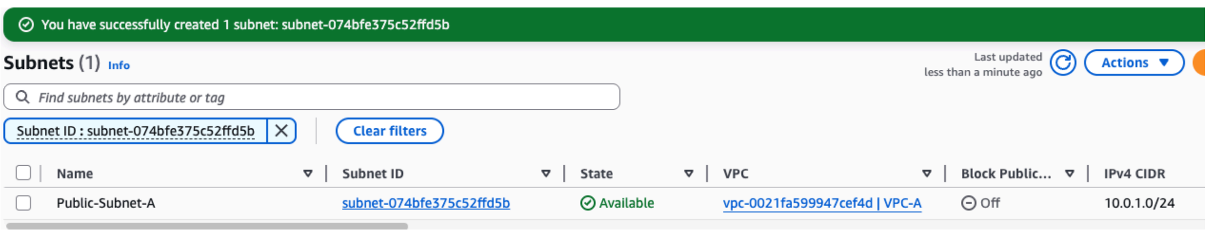
- **Subnets** → **Create subnet** yoluna gidin.



- **VPC ID**: `VPC-A`
- **Subnet name**: `Public-Subnet-A`
- **Availability Zone:** *No Preference*
- **IPv4 subnet CIDR block** : `10.0.1.0/24`



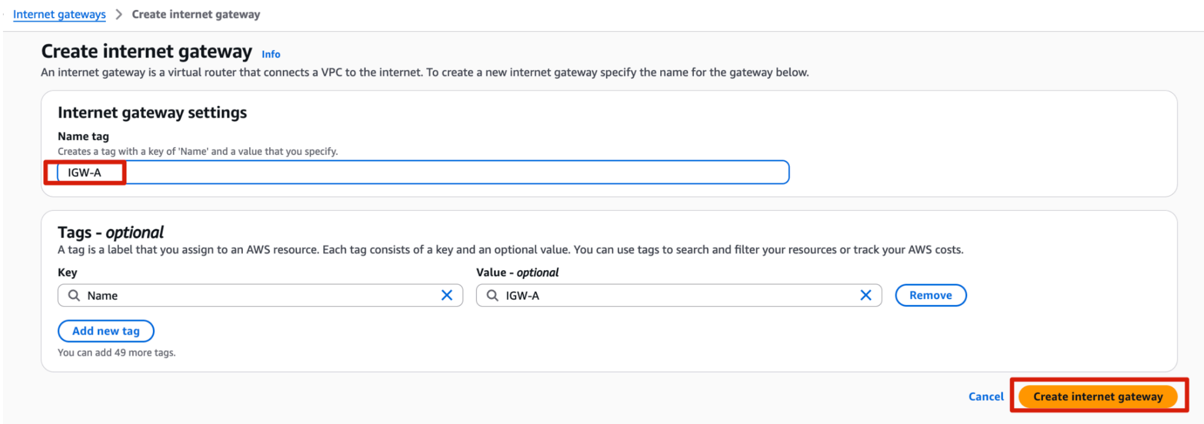
- AZ tercihini varsayılan bırakıp **Create subnet** butonuna basın.



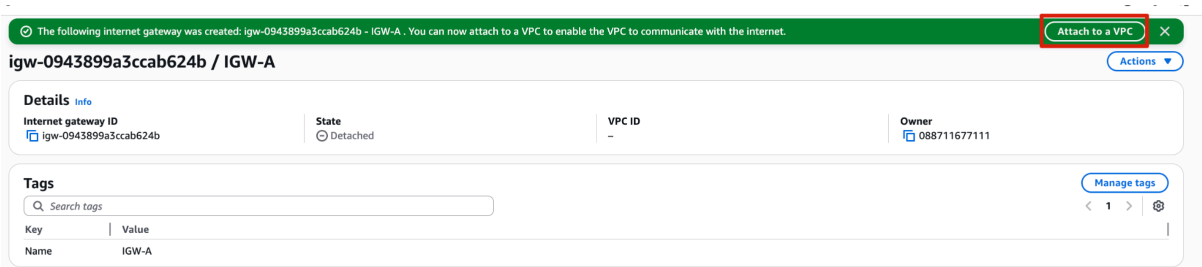
3. Internet Gateway Ekleme

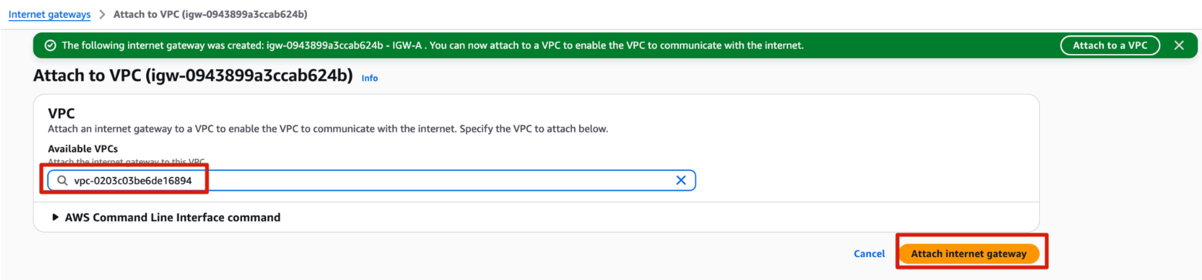
Public subnetteki EC2'nun internete çıkabilmesi için **Internet Gateway (IGW)** gerekli.

- **Internet Gateways → Create internet gateway**
- **Name:** `IGW-A`



- Oluşturduktan sonra **Attach to VPC → VPC-A** seçin.

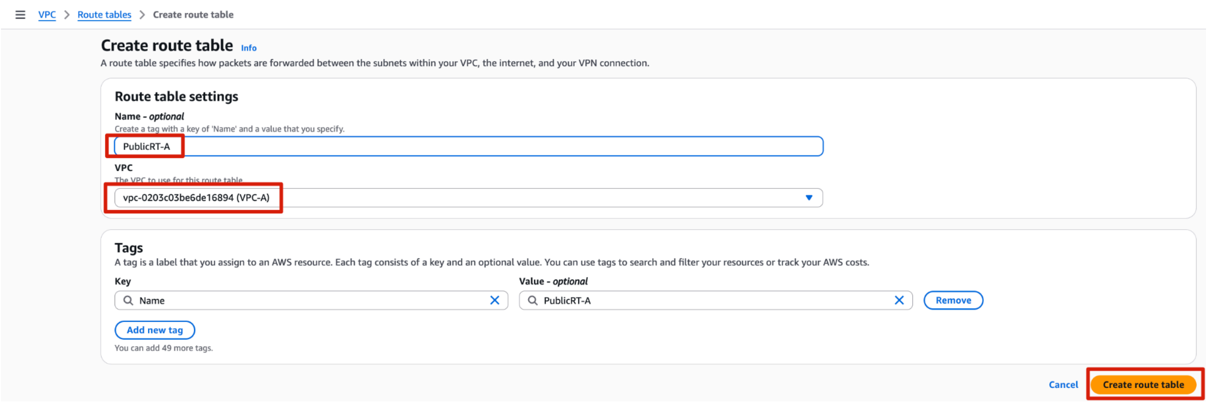




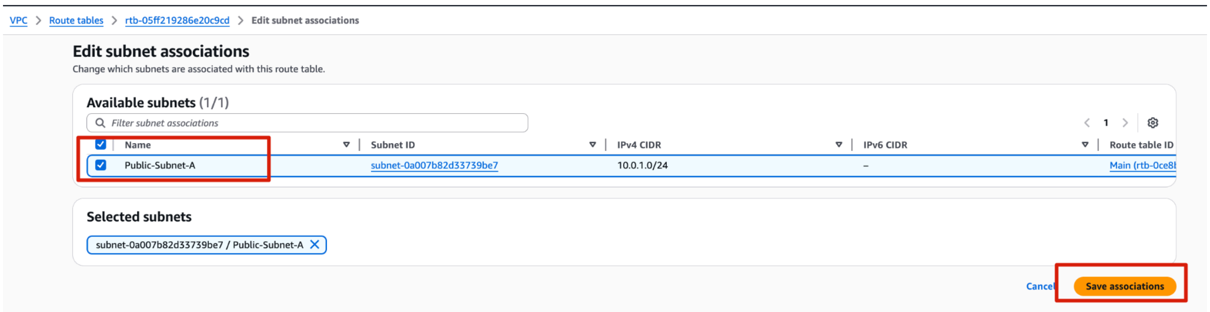
4. Public Route Table Hazırlama

Public subnet'in trafiğini IGW'ye yönetecek route table'ı oluşturacağız.

- **Route tables → Create route table**
- **Name:** `PublicRT-A`
- **VPC:** VPC-A

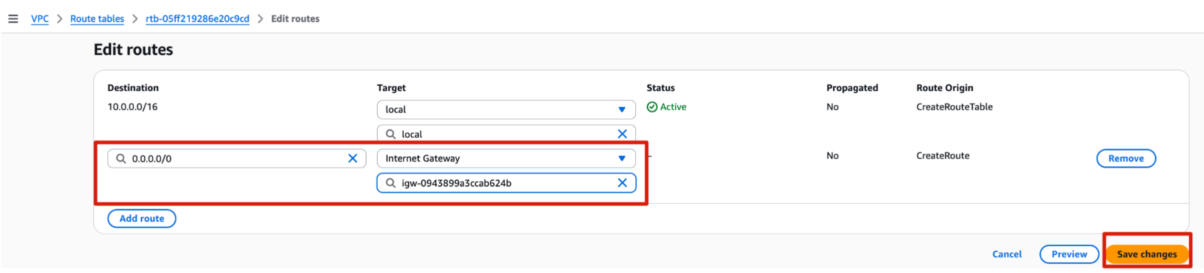


Route table'ı oluşturduktan sonra **Subnet associations → Public-Subnet-A** seçip kaydedin.



Edit routes → Add route diyerek şunu ekleyin:

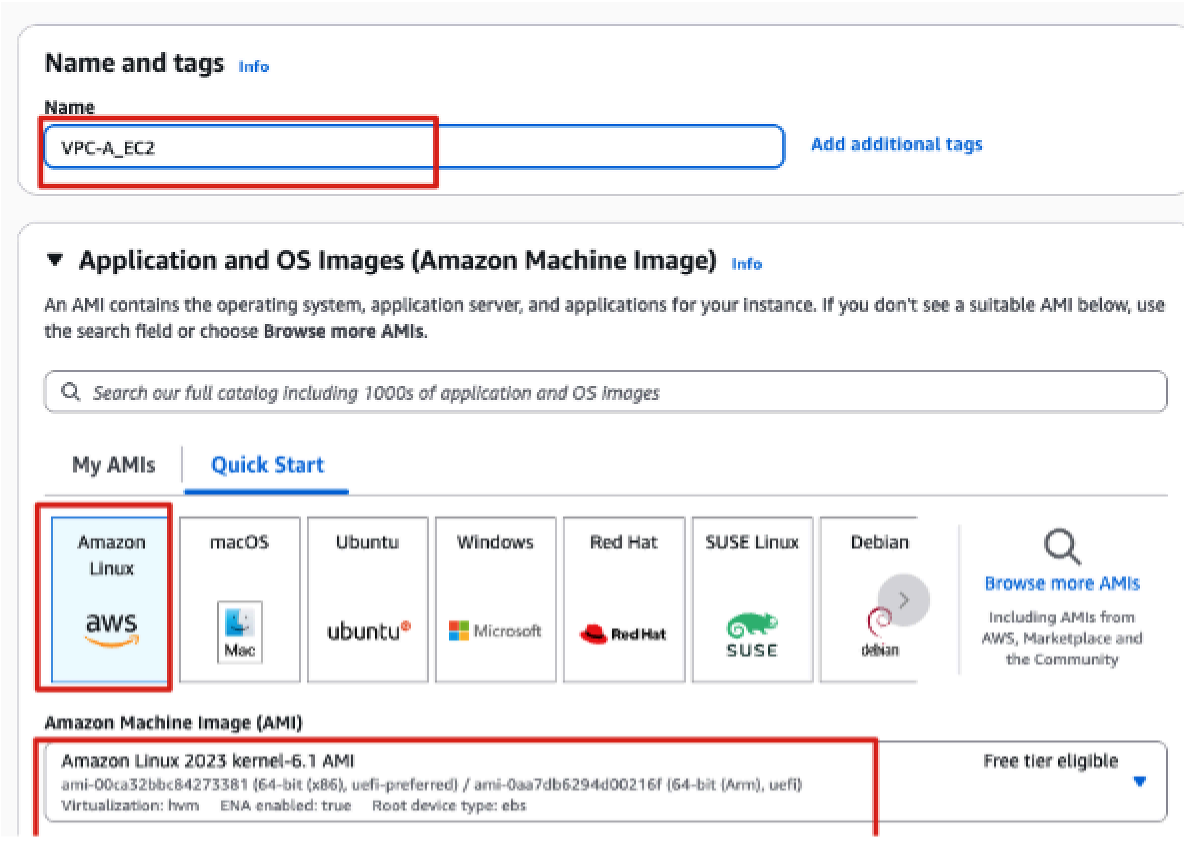
Destination: `0.0.0.0/0` **Target:** IGW-A



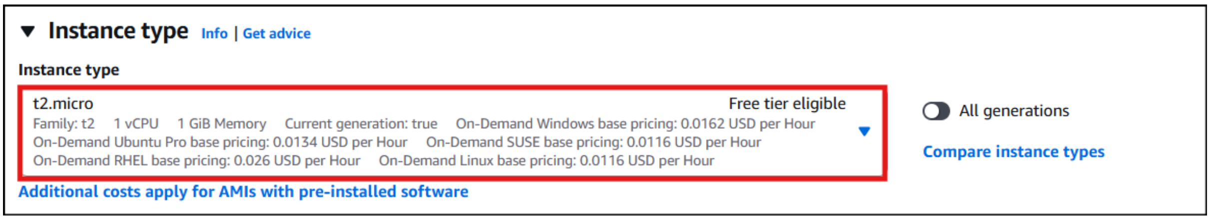
5. Bastion EC2 Oluşturma

Artık Public subnet içinde bir bastion host sunucusu açalım.

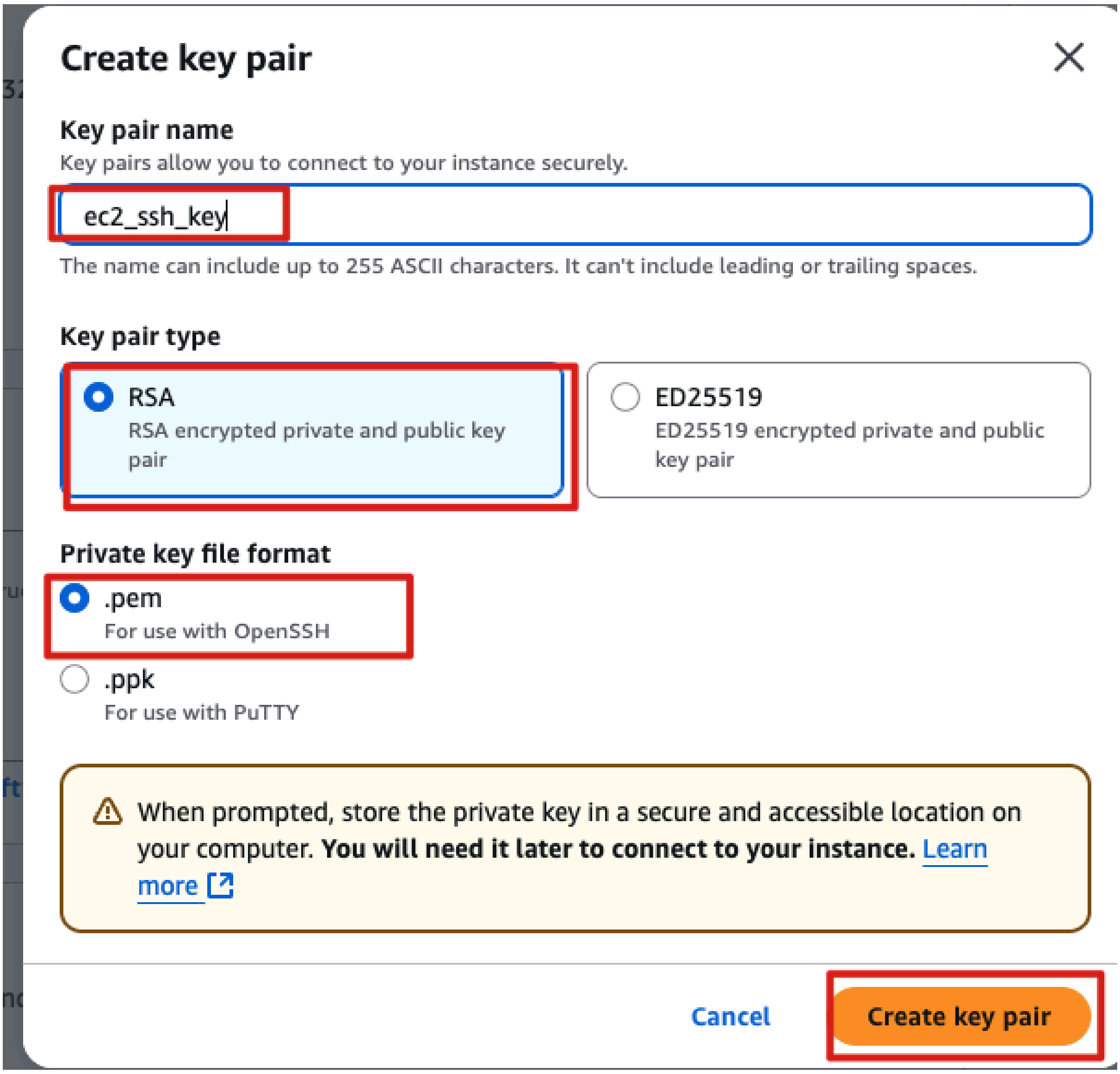
- Bölge: **us-east-1** (N. Virginia).
- **Services → EC2 → Instances → Launch instances**.
- **Name and tags:**
  - **Name:** `VPC-A_EC2`
- **Application and OS Images (AMI):**
  - **Quick Start → Amazon Linux 2023 (kernel-6.1)**



- **Instance type:** `t2.micro`



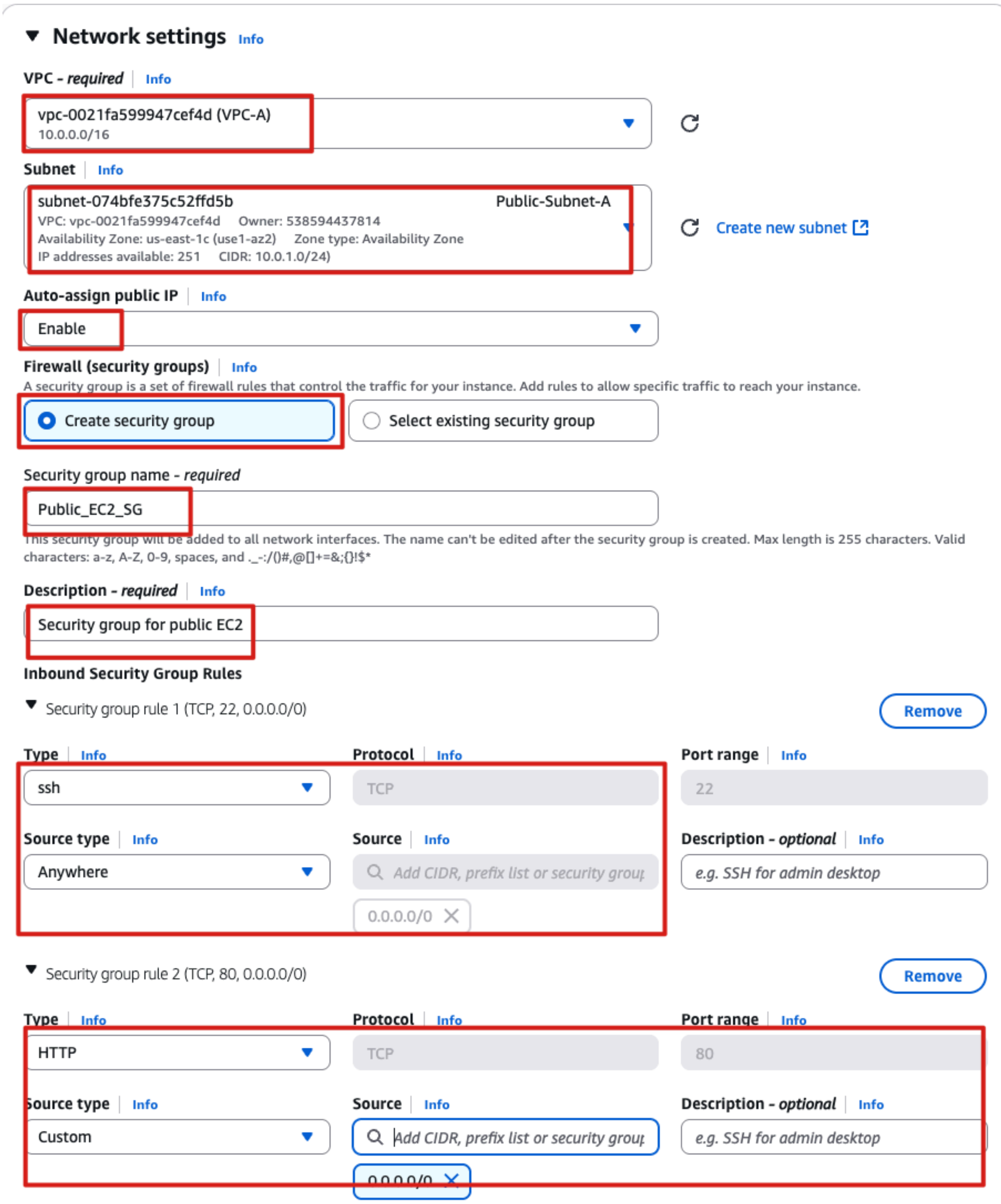
- **Key Pair (login):**
  - **Create new key pair → Key pair name:** `ec2_ssh_key`
  - **Type:** RSA, **File format:** `.pem` → **Create** ve seç.



Key dosyasını bilgisayarına kaydet

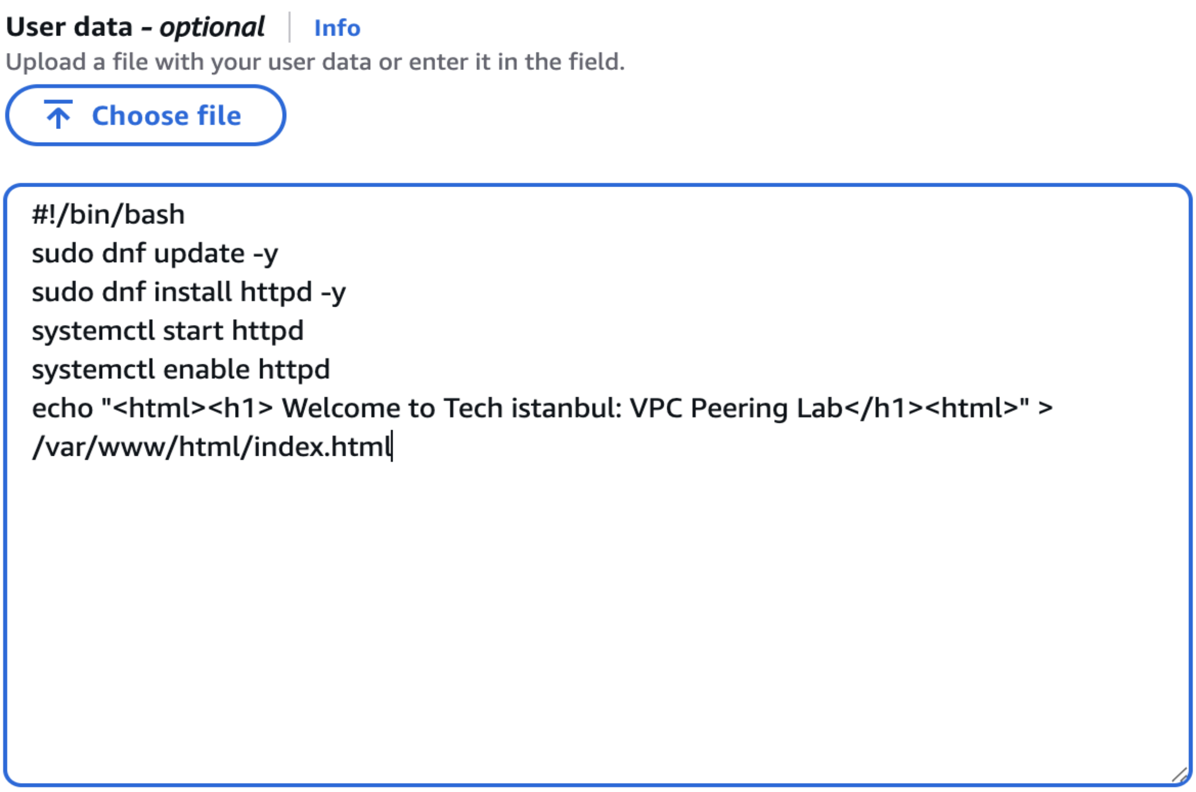
- **Network settings → Edit:**
  - **VPC:** `VPC-A`
  - **Subnet:** varsayılan (Public_subnet_VPC-A seçili gelmeli)
  - **Auto-assign public IP:** `Enable`
  - **Firewall (SG): Create a new security group**
    - **Security group name:** `Public_EC2_SG`
    - **Description:** `Security group for public EC2`

- **Rules:**
    1. **SSH** → **Source:** Anywhere (0.0.0.0/0)
    2. **HTTP** → **Source:** Anywhere

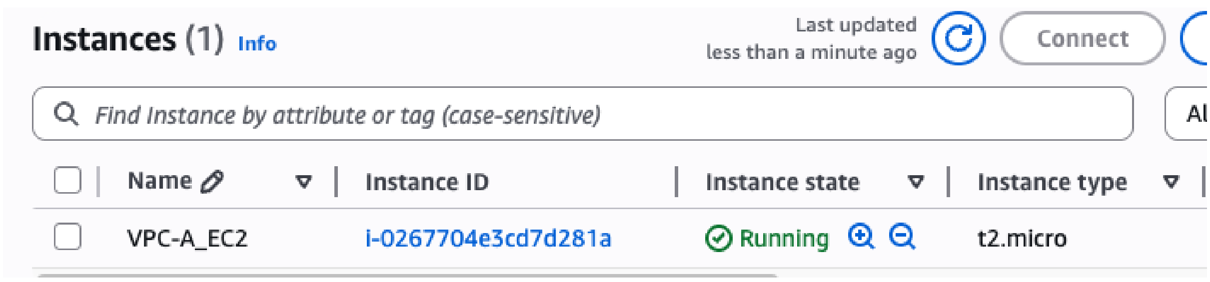


- **Advanced details:**
    - **User Data Script (opsiyonel, web sayfası göstermek için):**

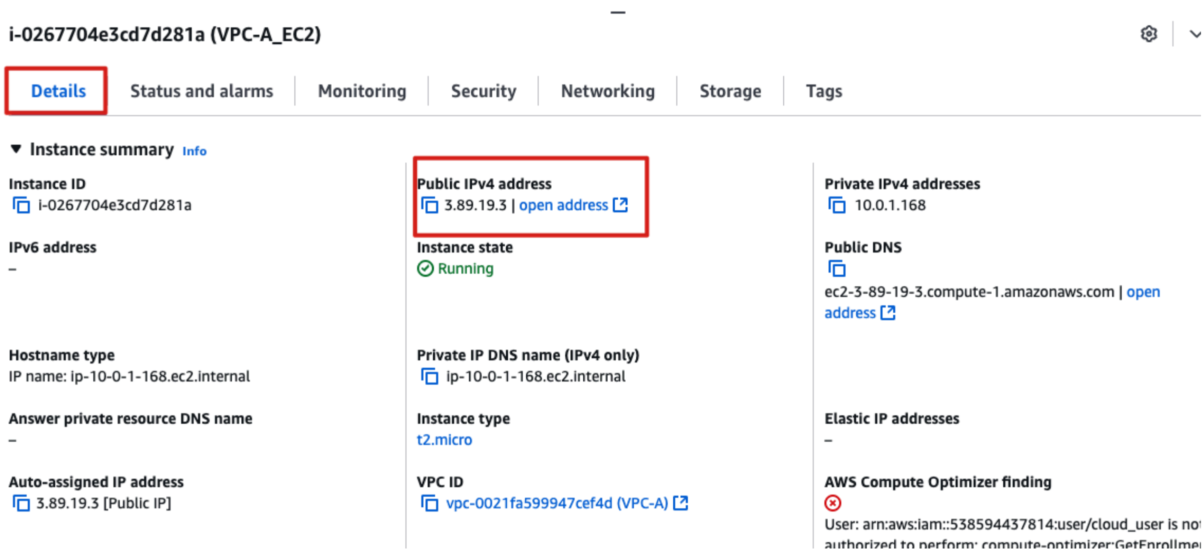
```bash
#!/bin/bash
sudo dnf update -y
sudo dnf install httpd -y
systemctl start httpd
systemctl enable httpd
echo "<html><h1> Welcome to Tech istanbul: VPC Peering Lab</h1><html>" >
/var/www/html/index.html
```



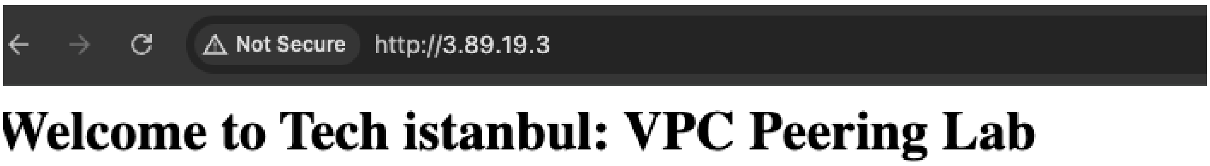
Launch instances butonuna tıkladığınızda, birkaç dakika içinde bastion sunucunuz hazır olacaktır. Public IP'yi tarayıcıya yapıştırarak test sayfasını görebilirsiniz.



- **EC2’nun IPv4 Public IP** adresini not et.



- Bu IP’yi tarayıcıya yapıştırdığında karşılama sayfasını görmelisin.

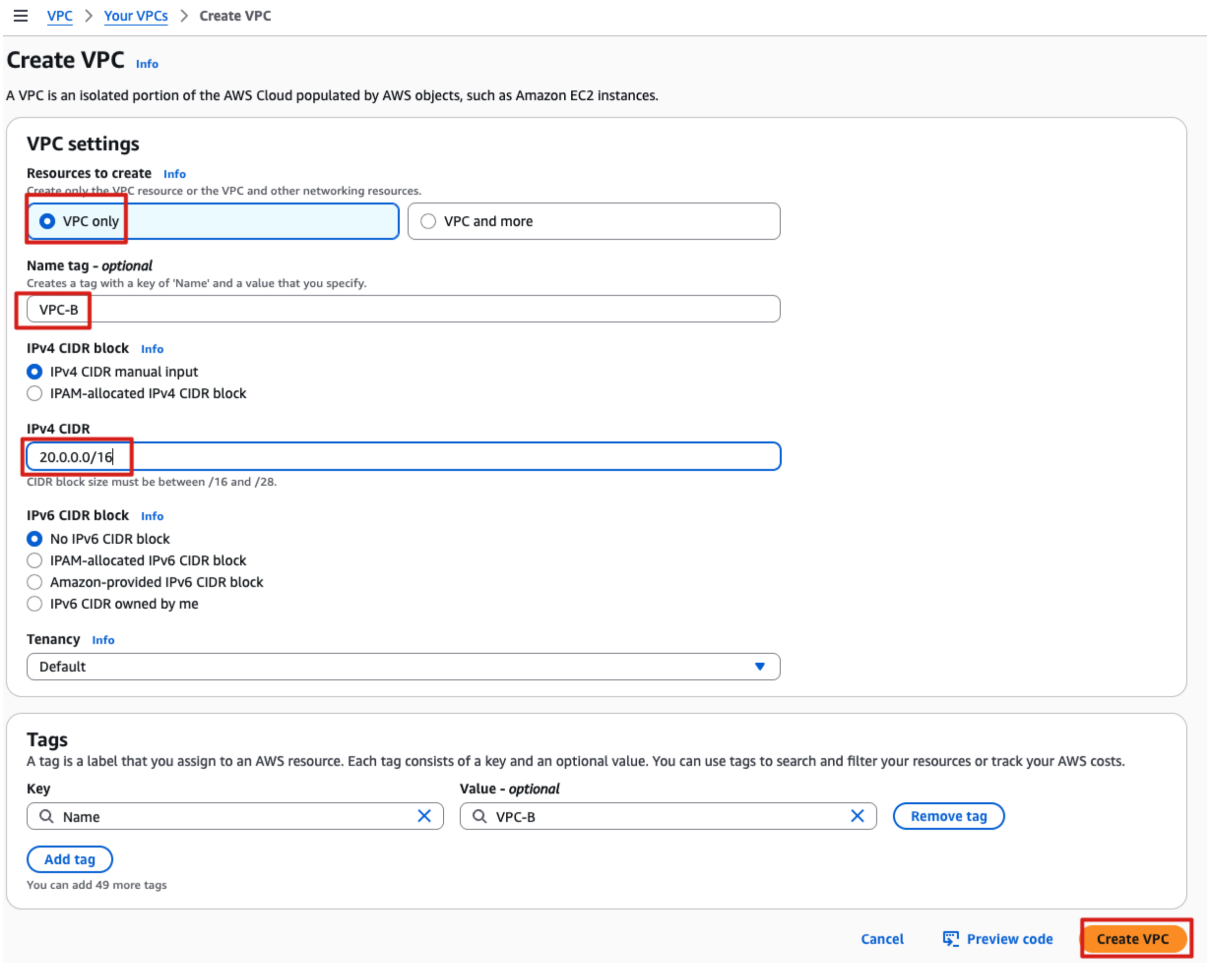


Bölüm 2: VPC-B (Private) Kurulumu

Şimdi ikinci VPC'mizi, yani **VPC-B**'yi oluşturacağız. Bu VPC'de yalnızca **private subnet** olacak ve EC2 instance'ın **public IP'si olmayacak**. Yani sadece içeriden, bastion üzerinden erişilebilir olacak.

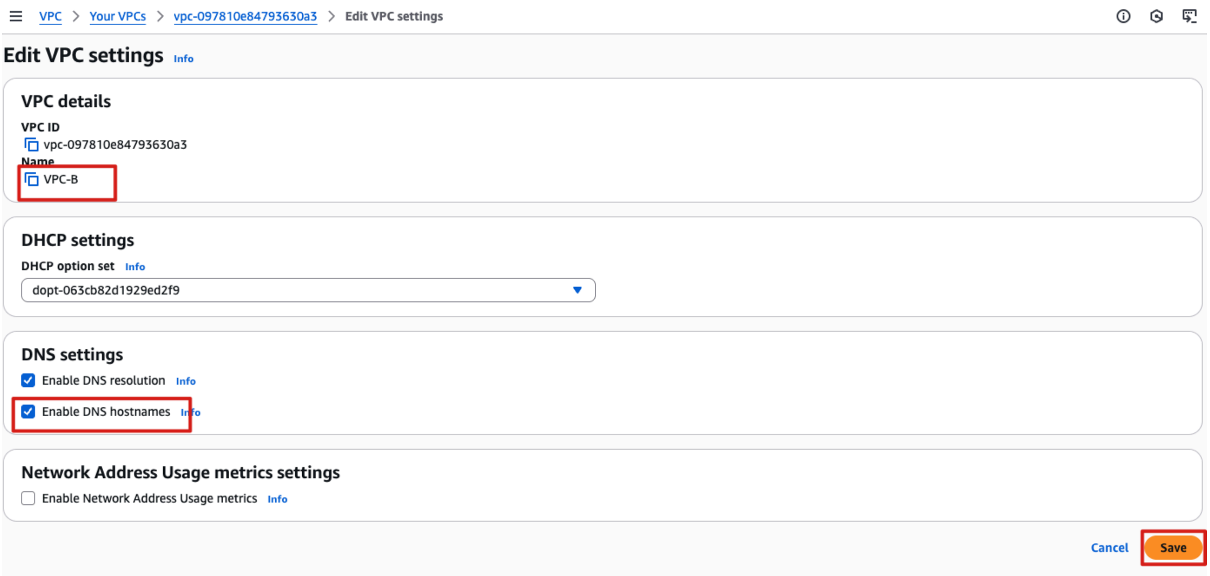
1. VPC Oluşturma

- **VPC → Your VPCs → Create VPC** yolunu izleyin.
- **Seçenek:** *VPC only*
- **Name:** `VPC-B`
- **IPv4 CIDR :** `20.0.0.0/16`
- **Create VPC** butonuna basın.



✅ Artık ikinci VPC hazır.

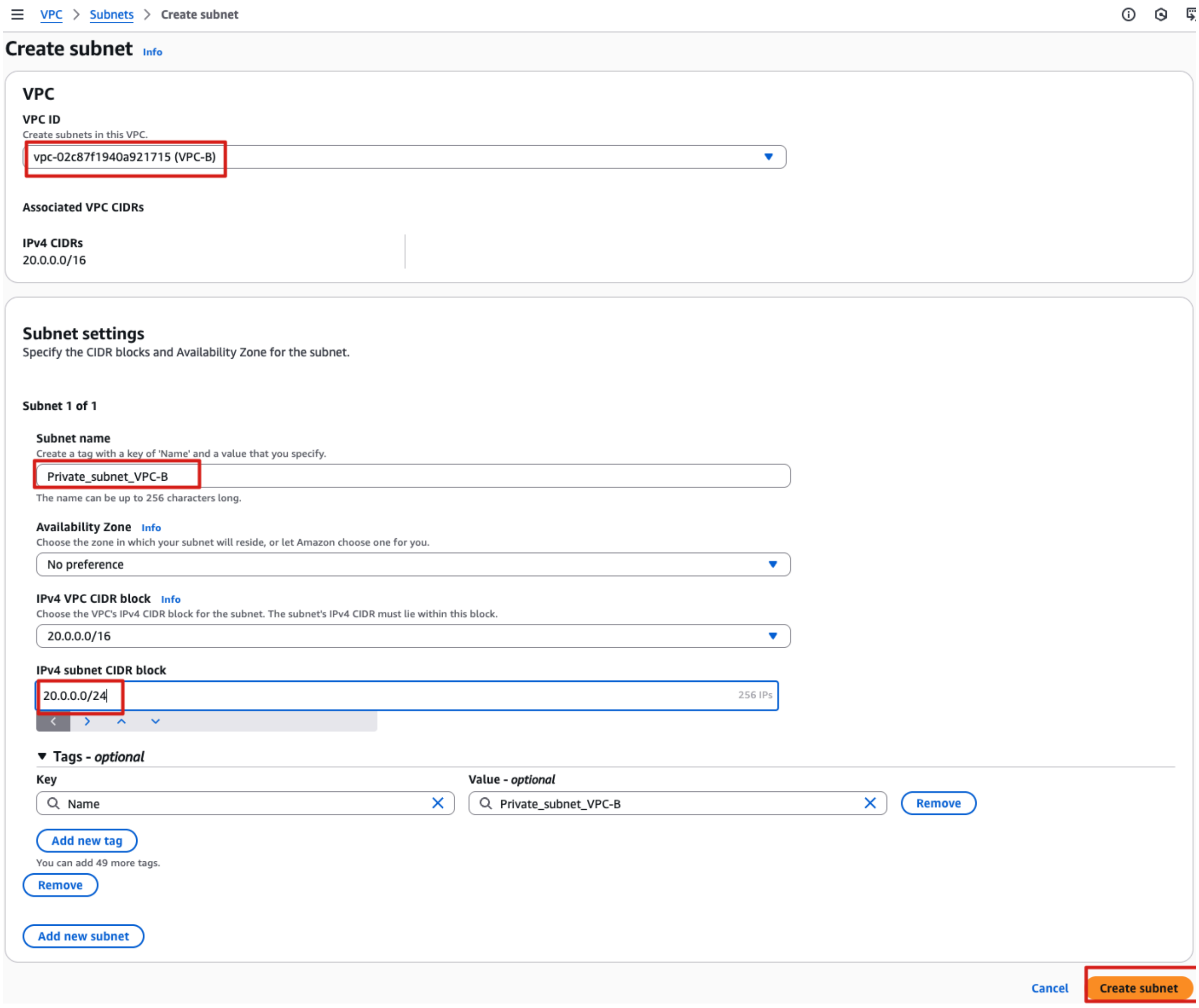
- Listeden **VPC-B** → **Actions** → **Edit VPC settings** →
  Enable DNS resolution ve **Enable DNS hostnames** işaretle → **Save**.



2. Private Subnet Oluşturma

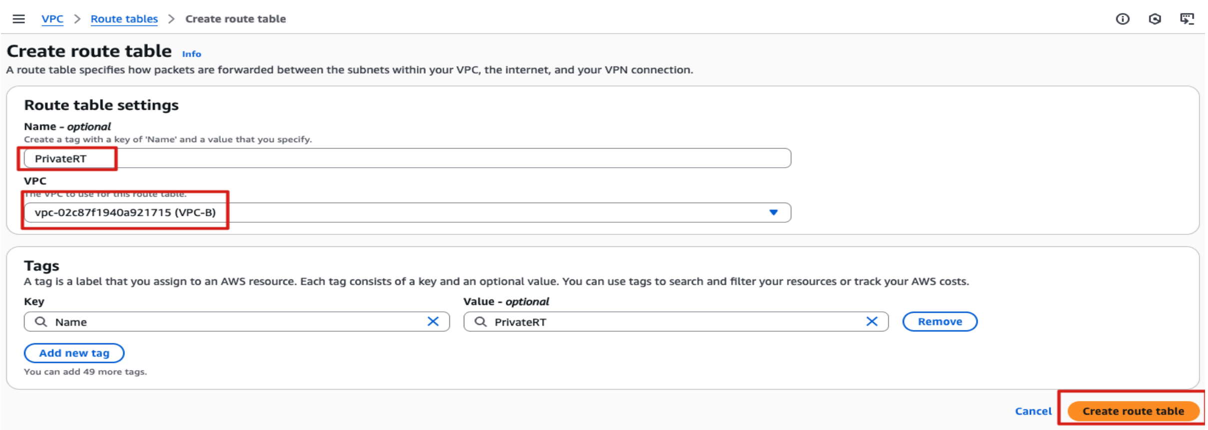
Bu subnet internete çıkmayacak, yalnızca iç iletişim için kullanılacak.

- **Subnets** → **Create Subnet** seçin.
- **VPC ID:** `VPC-B`
- **Subnet name:** `Private_subnet_VPC-B`
- **Availability Zone:** *No Preference*
- **IPv4 CIDR block:** `20.0.0.0/24`
- **Create subnet**.



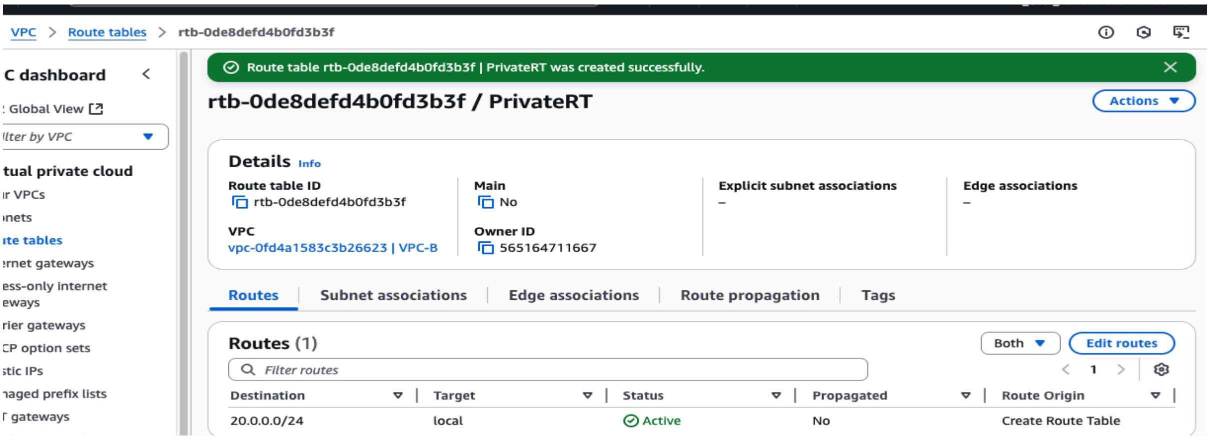
3. Private Route Table Hazırlama

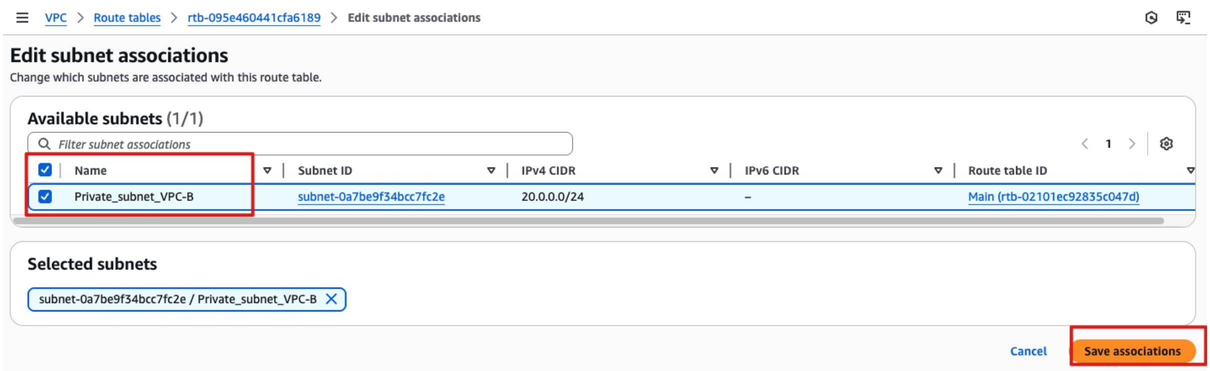
- **Route tables → Create route table**
- **Name:** `PrivateRT`
- **VPC:** `VPC-B`



- Route table oluşturduktan sonra, **Subnet associations** → **Private-Subnet-B** seçip kaydedin.

(Not: Burada IGW eklemiyoruz çünkü subnet'in internete çıkışı olmayacak.)

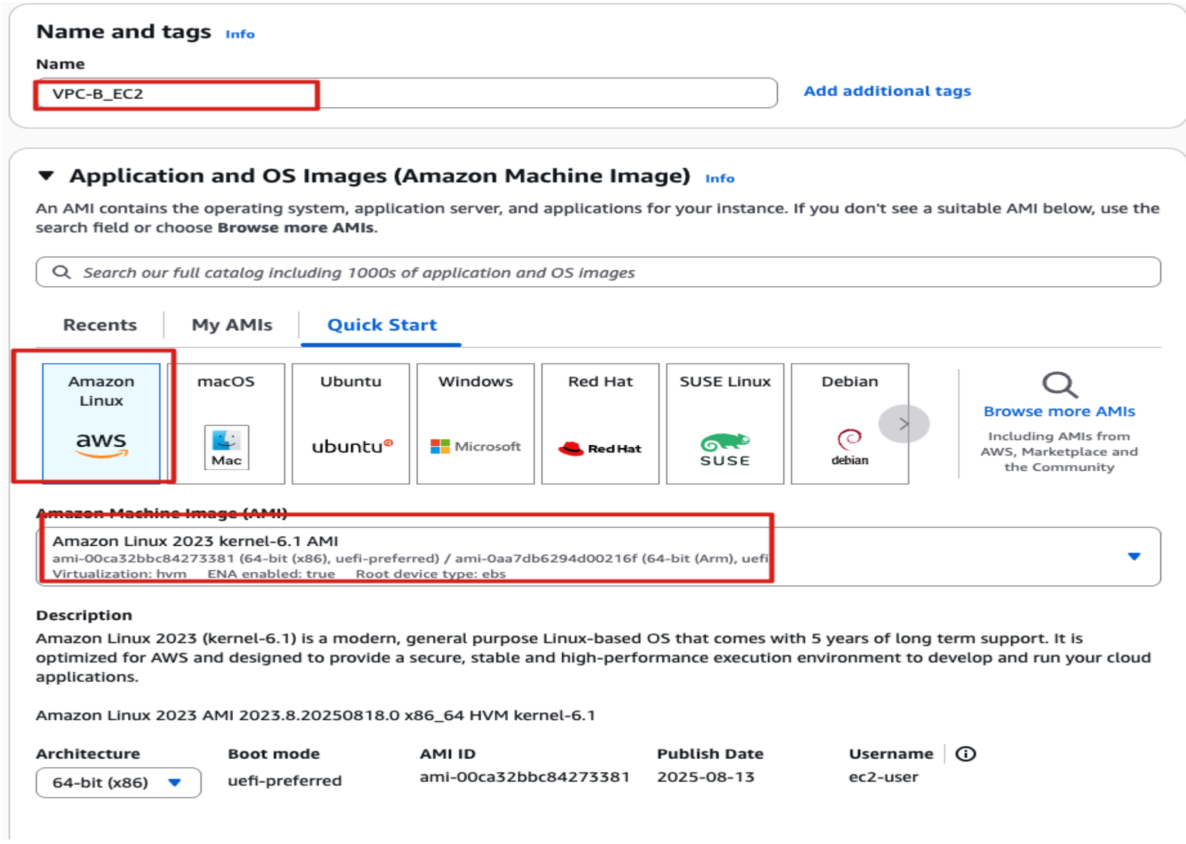




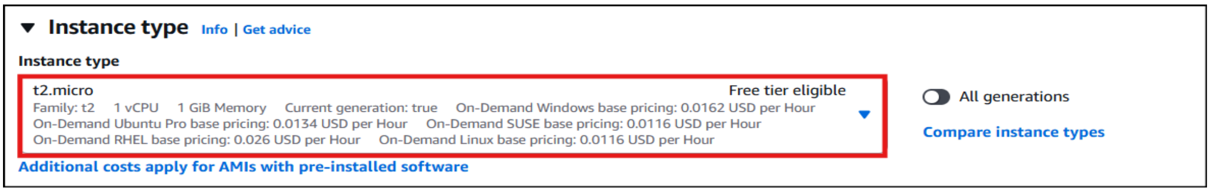
4. Private EC2 Oluşturma

Şimdi private subnet içinde internet erişimi olmayan bir EC2 başlatacağız.

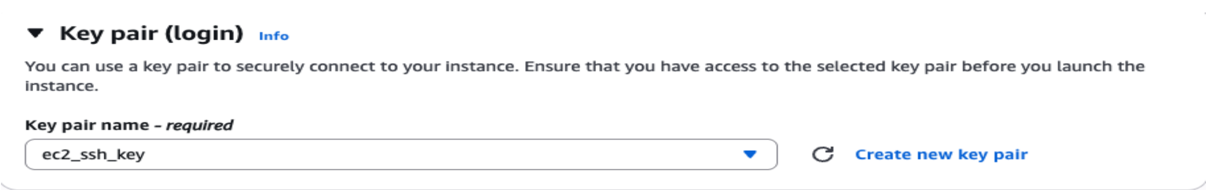
- **EC2** → **Launch instances**
- **Name:** `VPC-B_EC2`
- **AMI:** Amazon Linux 2023 (kernel-6.1)



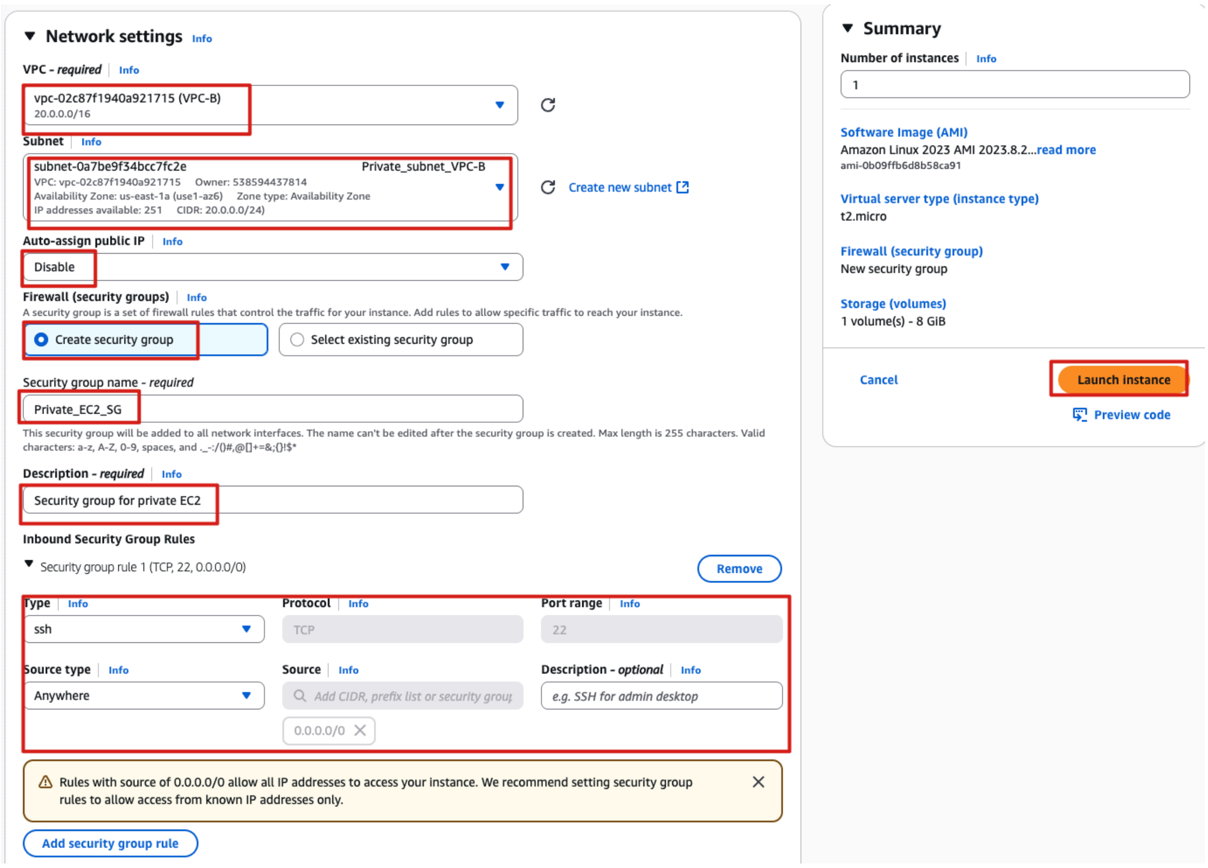
- **Instance type:** `t2.micro`



- **Key pair: Daha önce oluşturduğunuz `ec2_ssh_key.pem` dosyasını seçin.**

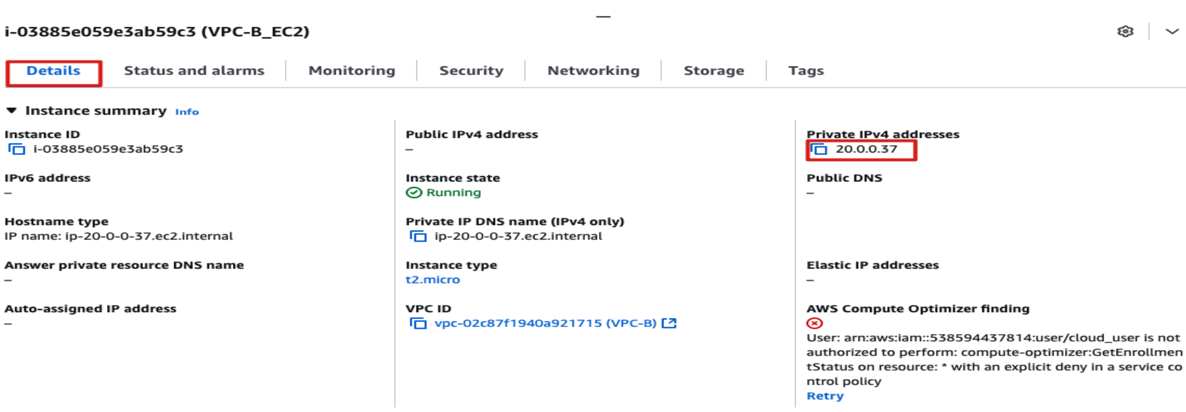


- **Network settings → Edit:**
  - **VPC:** `VPC-B`
  - **Subnet:** `varsayılan (Private_subnet_VPC-B)`
  - **Auto-assign public IP:** `Disable`
  - **Firewall (SG): Create a new security group**
    - **Name:** `Private_EC2_SG`
    - **Description:** `Security group for private EC2`
    - **Rule: SSH → Source:** Anywhere (lab için)



- **Launch instance** düğmesine bas ve → **running** olana kadar bekle.

✅ Bu instance yalnızca **Private IP** alacak. IPv4 Private IP adresini not edin (örneğin: `20.0.1.25`).



Bölüm 3: İki VPC Arası Bağlantıyı Test Et (Peering Öncesi)

Şimdi elimizde:

- **VPC-A_EC2 (Public bastion)** → Public IP'si var, dışarıdan erişilebilir.
- **VPC-B_EC2 (Private)** → Sadece private IP'ye sahip, internete çıkışı yok.

Amacımız, bastion üzerinden private EC2'ya **özel IP ile SSH** yapabilmek. Ama henüz peering olmadığı için bu bağlantı gerçekleşmeyecek.

1. Bastion'a Bağlan

Önce bilgisayarımızdan bastion sunucuya (VPC-A_EC2) SSH ile bağlanalım:

```
chmod 400 ec2_ssh_key.pem
ssh -i ec2_ssh_key.pem ec2-user@<VPC-A_EC2_PUBLIC_IP>
```

👉 Bu komutla bastion'a giriş yapıyoruz.

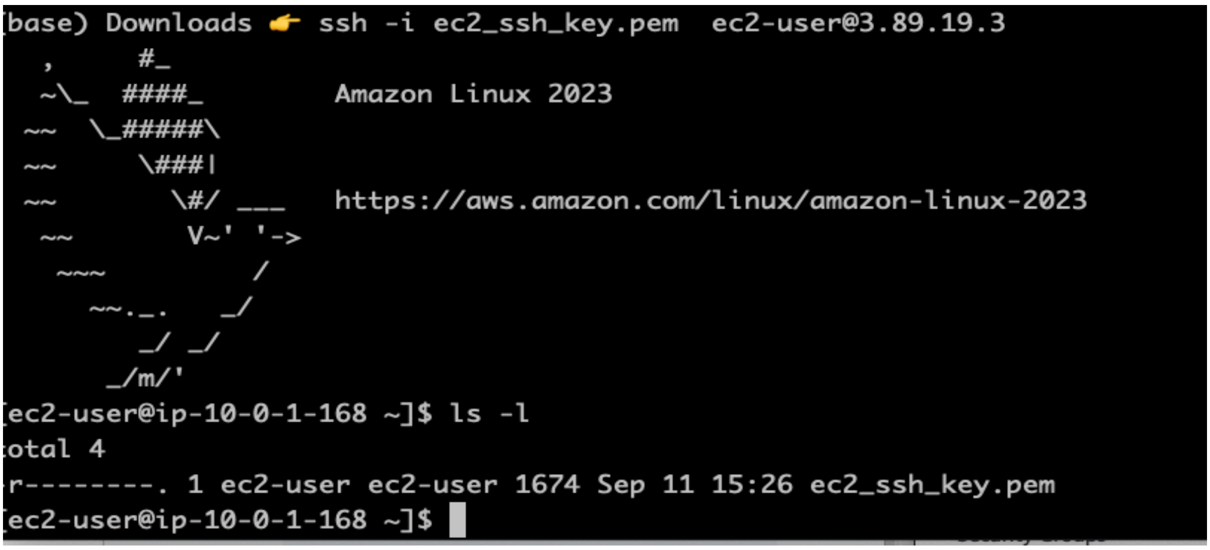
2. Key Dosyasını Bastion'a Kopyala

Private EC2'ya da aynı key ile bağlanacağımız için, `.pem` dosyasını localimizden bastion'a göndermeliyiz:

```
scp -i ec2_ssh_key.pem ec2_ssh_key.pem ec2-user@3.89.19.3:/home/ec2-user
```

Bastion'a ssh ile giriş yaptıktan sonra dosya izinlerini sıkılaştırın:

ssh -i ec2_ssh_key.pem ec2-user@3.89.19.3



```
chmod 400 ec2_ssh_key.pem
```

3. Private EC2’ya Bağlanmayı Dene

Artık bastion üzerinden private EC2’nun **private IP’si** ile SSH denemesi yapalım (örneğin `20.0.1.25`):

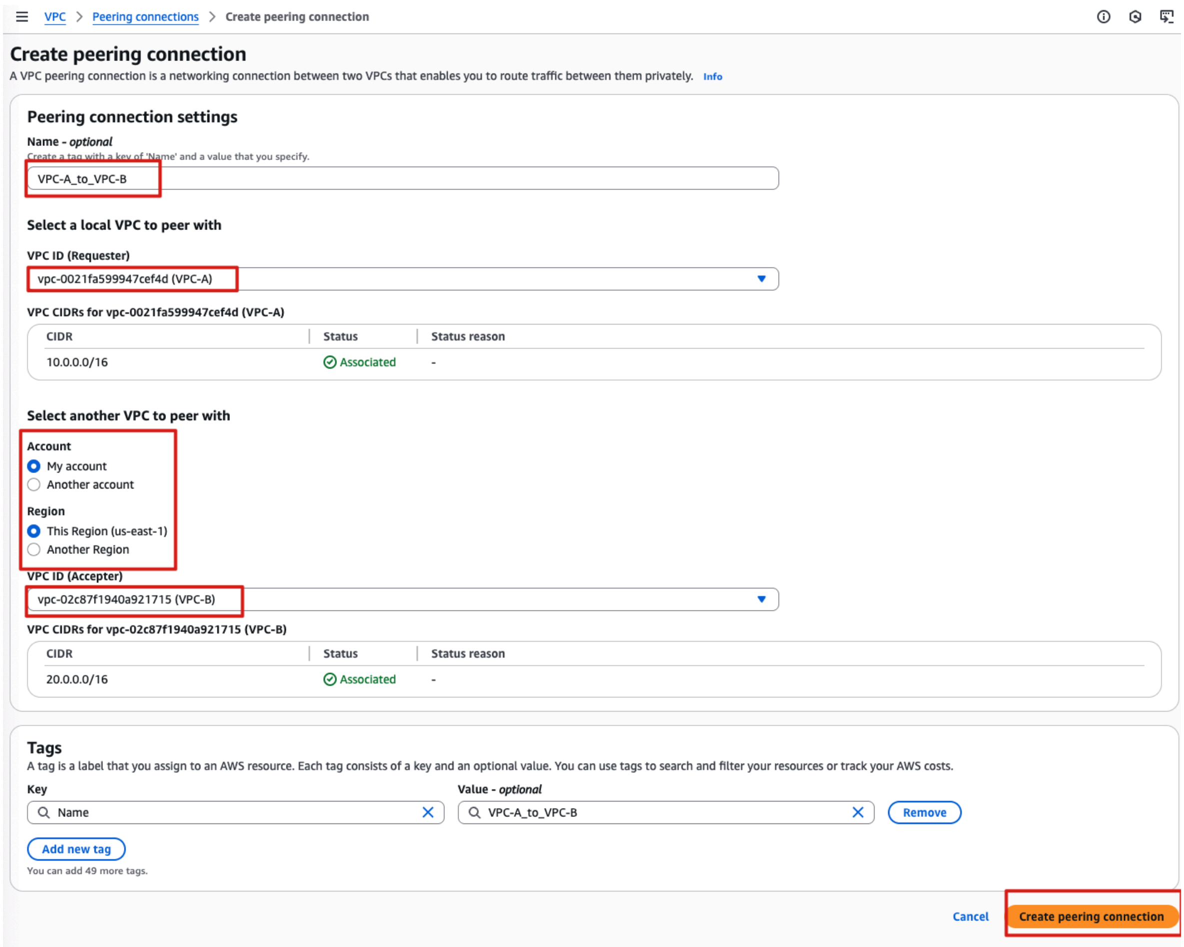
```
ssh -i ec2_ssh_key.pem ec2-user@20.0.1.37
```

✅ Bu noktada bağlantı **başarısız olacaktır**. Çünkü iki VPC arasında henüz bir iletişim yolu (peering) yok.

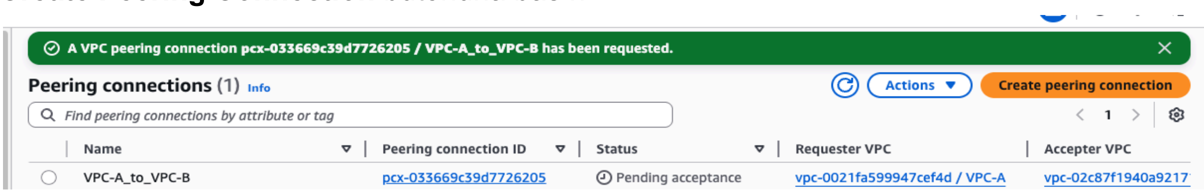
Bölüm 4: VPC Peering Oluşturma ve Route Table Güncelleme

Şimdi sıra geldi iki VPC’yi konuşturacak köprüyü kurmaya: **VPC Peering Connection**.

- AWS Console’da **VPC** → **Peering Connections** → **Create Peering Connection** yoluna gidin.
- **Peering connection name tag:** `VPC-A_to_VPC-B`
- **VPC ID (Requester):** VPC-A
- **VPC ID (Accepter):** VPC-B
- **Region:** Aynı region seçili olacak (ör. us-east-1).
- **Account:** Varsayılan bırakabilirsiniz.



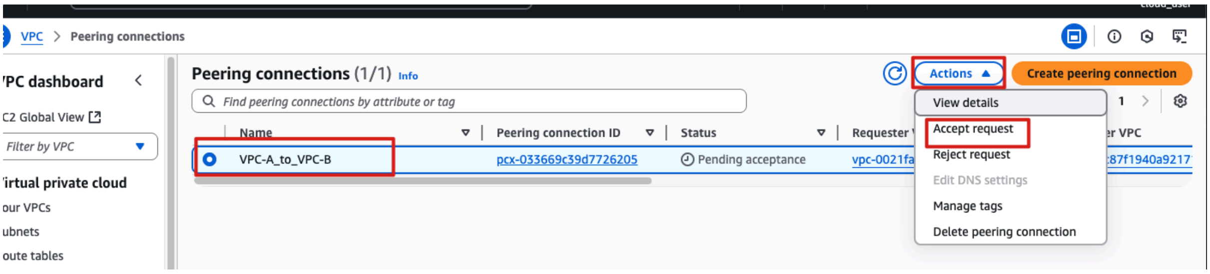
- **Create Peering Connection** butonuna basın.



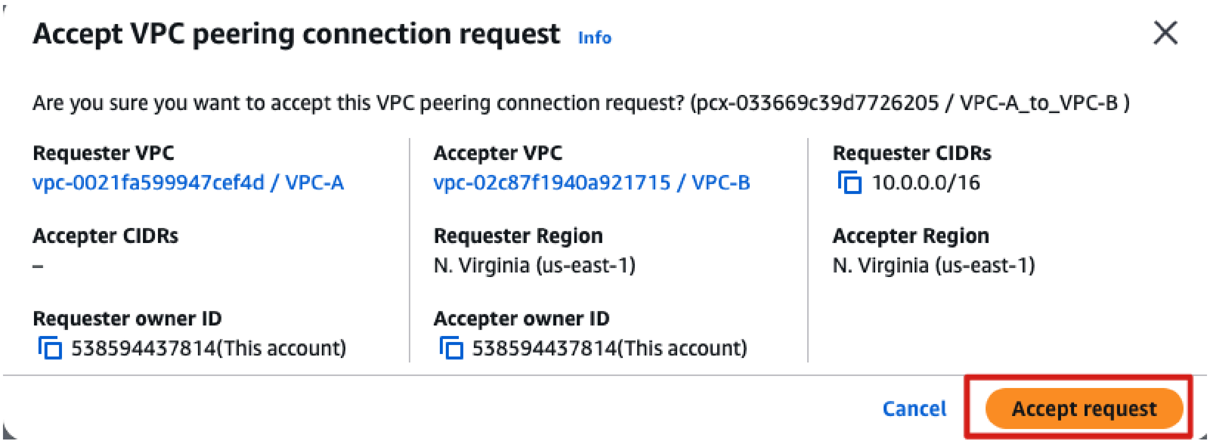
🔸 Durum ilk etapta **Pending Acceptance** olacaktır.

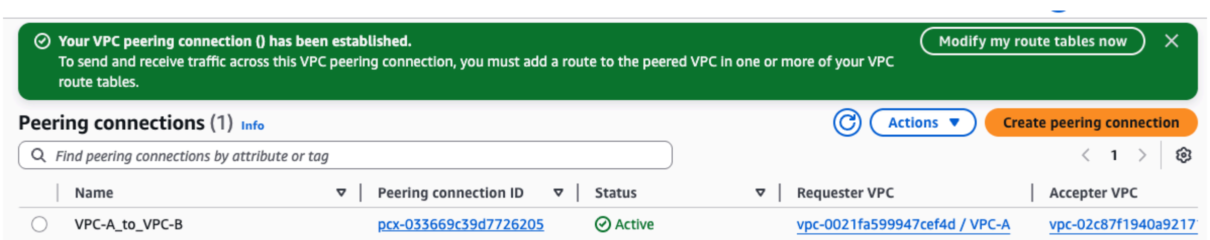
2. Peering İsteğini Kabul Etme

- **VPC → Peering Connections** menüsünden oluşturduğunuz bağlantıyı seçin.
- **Actions → Accept request** tıklayın.
- Artık peering'in durumu **Active** olmalı.



Durum **Active** olmalı (görmüyorsan sayfayı yenile).



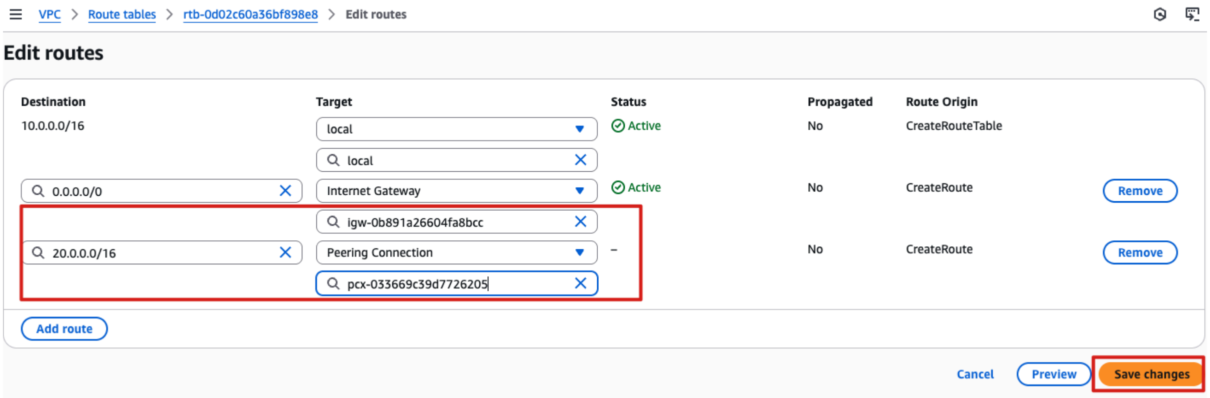


VPC-A dan VPC-B ye Peering **aktif** olsa da trafik ancak route ekleyince akar.

3. Route Table Güncellemeleri

Peering bağlantısı aktif olsa da, trafiğin akabilmesi için her iki VPC'nin route tablolarını güncellememiz gerekir.

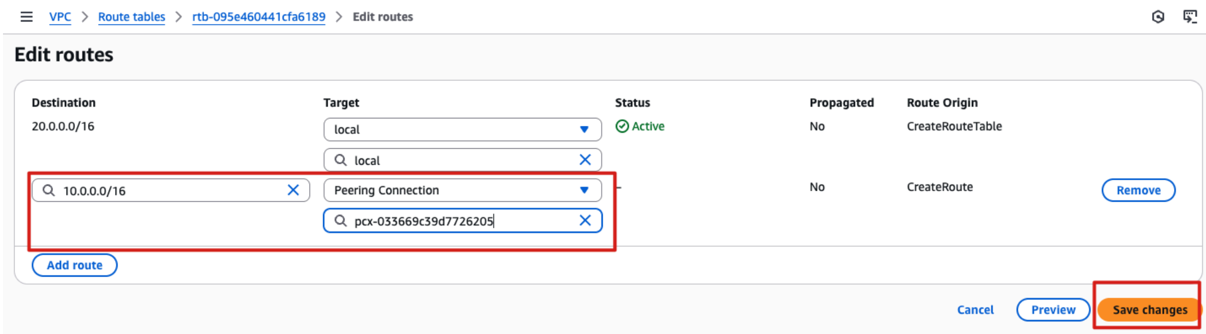
Route tables→PublicRT-A→edit routes



PublicRT-A (VPC-A):

- **Destination:** `20.0.0.0/16`
- **Target:** Peering connection (VPC-A_to_VPC-B)

Route tables→PrivateRT→edit routes



PrivateRT-B (VPC-B):

- **Destination:** `10.0.0.0/16`
- **Target:** Peering connection (VPC-A_to_VPC-B)

🔑 Bu sayede iki VPC arasındaki trafik karşılıklı yönlendirilecektir.

✅ **Artık VPC-A’daki bastion üzerinden VPC-B’deki private EC2’ya SSH yapabilirsiniz.**

```
ssh ec2-user@20.0.0.37 -i ec2_ssh_key.pem
```

```
[ec2-user@ip-10-0-1-168 ~]$  ssh ec2-user@20.0.0.37 -i ec2_ssh_key.pem
The authenticity of host '20.0.0.37 (20.0.0.37)' can't be established.
ED25519 key fingerprint is SHA256:vhjk8hi2HRzwlh4rxGHLC2M9Cdk9Cte6HMbqUgDztwE.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.0.0.37' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-20-0-0-37 ~]$ 
```

BÖLÜM 5: Kaynakları Temizleme (Cleanup)

Lab ortamlarında yaptığımız her kurulum, AWS üzerinde **gerçek kaynaklar** oluşturur ve bunlar dakikalar içinde fark etmeden maliyet yazmaya başlayabilir. Özellikle **EC2 instance** ve **VPC Peering Connection** gibi kaynaklar, uzun süre açık kalırsa gereksiz masraf çıkarır.

Bu nedenle lab bittikten sonra şu adımları takip ederek ortamı temizleyelim:

1. **EC2 Instance'ları Durdur ve Sil**
   - `VPC-A_EC2` (public bastion)
   - `VPC-B_EC2` (private sunucu)
2. **Subnet'leri Sil**
   - `Public-Subnet-A`
   - `Private-Subnet-B`
3. **Route Tablolarını Sil**
   - `PublicRT-A`
   - `PrivateRT`
4. **Internet Gateway'i Sil**
   - `IGW-A`
5. **VPC Peering Connection'ı Sil**
   - `VPC-A_to_VPC-B`
6. **VPC'leri Sil**
   - `VPC-A`
   - `VPC-B`

🎯 Sonuç

Bu lab'da öğrendiklerimizi özetleyelim:

- İki VPC arasında güvenli iletişim kurmak için **VPC Peering** oluşturduk.
- **Public Bastion** → **Private EC2** üzerinden SSH bağlantısını test ettik.
- Peering'in çalışabilmesi için **route tablolarını çift taraflı güncelledik**.
- Lab sonunda kaynakları temizleyerek maliyetleri sıfırladık.

VPC Peering; basit yapısıyla küçük/orta ölçekli senaryolar için idealdir. Ancak çok sayıda VPC'yi birbirine bağlamak gerektiğinde **Transit Gateway** gibi daha merkezi çözümler tercih edilebilir.